



OSTBAYERISCHE  
TECHNISCHE HOCHSCHULE  
REGENSBURG

ELEKTRO- UND  
INFORMATIONSTECHNIK

# **PRAKTIKUM INFORMATIK 1**

M. Niemetz

## **Algorithmen II**

Sortieralgorithmen

Dezember 2025, Version: 1.85

Fakultät Elektro- und Informationstechnik

# 1 Einleitung

Sortieren ist eine der häufigsten Aufgaben in der Softwareentwicklung, da sortieren ein Basiselement für sehr viele Algorithmen für unterschiedlichste Probleme darstellt bzw. eine sortierte Menge für die Algorithmen die Voraussetzung darstellt.

In dieser Aufgabe werden Sie verschiedene Sortieralgorithmen umsetzen und sie anhand ihrer zeitlichen Performance vergleichen.

Dabei sollen Sie erkennen, welche Bedeutung eine gute Auswahl des Algorithmus für die Performance der Software auch auf schnellen modernen Computersystemen hat.

Dabei geht es nämlich nicht nur um eine kurze Ausführungszeit, sondern auch darum, die Erledigung der (Sortier-) Aufgabe mit möglichst geringer Ressourcennutzung – also möglichst nachhaltig zu erledigen.

So benötigen ineffiziente Implementierungen zur Ausführung in akzeptabler Zeit nicht nur einen aufwendigeren (teureren) Prozessor mit mehr Chipfläche und höheren Taktraten, sondern insbesondere auch mehr Energie für die Herstellung und den Betrieb des Systems.

## 2 Vorbereitungen

### 2.1 Funktionspointer

Da wir im Verlauf der weiteren Aufgabe verschiedenste Sortieralgorithmen testen und vergleichen wollen, brauchen wir eine Möglichkeit, diese verschiedenen Funktionen in Schleifen so aufzurufen, dass in jedem Schleifendurchlauf eine andere Funktion aufgerufen wird. Stellen Sie sich das analog dazu vor, wie Sie dies von Datenelementen in Arrays bereits kennen, wo auch häufig in jedem Schleifendurchlauf ein anderes Element des Array verarbeitet bzw. beschrieben wird.

Während man Daten bzw. Pointer auf Daten leicht in Arrays packen und so per Index darauf zugreifen kann, ist die Lösung für Funktionen nicht so offensichtlich.

Aber auch hier ist das Prinzip der Pointer anwendbar, da ja der ausführbare Code der Funktionen später auch irgendwo im Arbeitsspeicher des Rechners liegen muss. So wie es also Pointervariablen gibt, welche Adressen von Datenelementen speichern, gibt es auch Pointervariablen, die Adressen von ausführbarem Code speichern, die sogenannten Funktionspointer (also Pointer auf Funktionen).

Die Syntax der Typbeschreibung für Funktionspointer ist etwas sperrig, aber nicht schwierig. Eine Variablendefinition für eine Variable, die einen

Funktionspointer aufnehmen kann, sieht so aus (die markierten Bereich sind entsprechend der eigenen Situation zu ersetzen):

```
Rückgabetyt (* PointerVariablenname )( typ1 Arg1, ... );
```

also zum Beispiel:

```
void (*doMySorting)(int Data[], long count);
```

So ein Pointer kann dann die Adresse einer Funktion aufnehmen. Dabei ist nur darauf zu achten, dass die Signatur der Funktion (die Anzahl und die Typen der Parameter sowie der Rückgabetyt) auch exakt zum Typ des Pointer passen:

```
void ThisIsMySortingFunction(int Data[], long count)
{
    //This implements LazySort, doing simply nothing,
    //maybe the result is accidentally sorted?
}
```

Um nun die Adresse der Funktion `ThisIsMySortingFunction` im Pointer `doMySorting` zu speichern, genügt es zu schreiben:

```
doMySorting=ThisIsMySortingFunction;
```

Hier erkennt man, dass der **Name einer Funktion** einen **Pointer auf diese Funktion repräsentiert**. Die üblichen **runden Klammern** dahinter stellen dann den „**Funktionsaufrufsoperator**“ dar, welcher sicherstellt, dass die Funktion ausgeführt und dabei die Parameter in den Klammern übergeben werden.

Der Funktionspointer kann dann genutzt werden, um die Funktion aufzurufen:

```
doMySorting(MyData, MyDataCount);
```

Um die Verwendung etwas übersichtlicher zu gestalten, kann man natürlich auch, wie von den übrigen Datentypen gewohnt, einen eigenen Typnamen vergeben:

```
typedef Rückgabetyt (* NeuerTypname )( typ1 Arg1, ... );
```

Also:

```
typedef void (*MySortFunctionType)(int Data[], long count);
```

Damit wird dann die Definition der Variable deutlich übersichtlicher:

```
MySortFunctionType doMySorting;
```

Verwendung finden Funktionspointer in vielfältiger Weise, zum Beispiel als Parameter in Funktionen. Selbstverständlich kann man diesen Typ auch verwenden, um Arrays von Funktionspointern zu definieren, wodurch die Funktionspointer äußerst nützlich werden, da man die Möglichkeit hat, über verschiedene Funktionen in einer Schleife zu iterieren.

## 2.2 Der Projektrahmen

Damit Sie bei der Umsetzung dieser Aufgabe etwas Zeit sparen, haben Ihre Betreuer für Sie vorgearbeitet und einen kleinen Programmrahmen vorbereitet, der Ihnen folgende Dinge liefert:

- Eine **Beispielimplementierung von Bubblesort**, die Ihnen zeigen soll, wie Sie Ihre eigenen Sortieralgorithmen implementieren können und die als Versuchs- und Vergleichskandidatin dient.
- **Typdefinitionen:**

```
/* Type of data to be sorted */  
typedef double t_sortdata;  
/* Type for function pointers to sort functions */  
typedef void (*SortAlgorithm)(t_sortdata Data[],  
                             long DataCount);
```

- Eine Funktion zum **Generieren von mit zufälligen Daten gefüllten Arrays**, der man einfach nur die gewünschte Menge an Daten übergibt. Diese kümmert sich selbst um die Reservierung des notwendigen Speichers und liefert als Rückgabeparameter einen Pointer auf den entsprechenden mit Zufallszahlen gefüllten Datenbereich des Arrays zurück:

```
t_sortdata* generateRandomData(long DataCount);
```

- Eine Funktion zum **Ausgeben eines solchen Arrays** auf die Konsole für Debug-Zwecke, um die korrekte Sortierung auch leicht kontrollieren zu können:

```
void printDataArray(t_sortdata Data[],  
                   long DataCount);
```

- Eine Funktion **swapdata** welche die **Werte zweier Speicherplätze** des Typs **t\_sortdata** vertauscht.

Zudem wird auch noch eine Beispieldatei mit einer **main**-Funktion mitgeliefert, die eine Beispielanwendung zusammen mit der bereits aus früheren Aufgaben bekannten Zeitmessbibliothek zeigt.

Laden Sie das Archiv mit dem Programmrahmen auf Ihren Rechner und kompilieren Sie das Programm.

Analysieren Sie dann die **main**-Funktion, damit Sie verstehen, wie die gelieferten Funktionen und Typen zu verwenden sind.



Bringen Sie das Beispielprogramm aus dem vordefinierten Programmrahmen zum Laufen und machen Sie sich mit dem Beispielprogramm vertraut.

## 2.3 Shakersort

Implementieren Sie einen zweiten Sortieralgorithmus namens *Shakersort* (siehe z.B. [1]). Dieser Algorithmus ist unter mehreren Namen wie zum Beispiel *Cocktailsort* oder auch *Doublebubble* bekannt.

Die Idee hierbei ist es, einen Nachteil von *Bubblesort* auszugleichen, der sich dadurch ergibt, dass die Liste der zu sortierenden Objekte immer in der selben Reihenfolge durchlaufen wird und sich dadurch die Elemente in einer Richtung schneller bewegen als in der anderen.

Bei *Shakersort* wird diese Asymmetrie dadurch gelöst, dass die Liste abwechselnd in der einen und dann in der anderen Richtung durchlaufen wird. Das bedeutet, die innere Schleife von *Bubblesort* ist zweimal, jedoch mit umgekehrter Iterationsrichtung, vorhanden.

Ansonsten folgt *Shakersort* der Idee von *Bubblesort*: Es werden immer zwei benachbarte Elemente verglichen und bei Bedarf vertauscht. Dies wird, wie bei *Bubblesort*, so lange fortgesetzt, bis ein ganzer Durchlauf keine Vertauschung mehr zur Folge hatte.

**Hinweis:** Verwenden Sie die bereits existierende Implementierung für *Bubblesort* als Vorlage für Ihre *Shakersort*-Implementierung, indem sie diese Funktion kopieren und modifizieren.

Um zu prüfen, ob Ihre Sortierfunktion auch tatsächlich korrekt sortiert, müssen Sie prüfen, ob das Array nach dem Aufruf der Sortierfunktion auch tatsächlich korrekt und vollständig sortiert ist. Solche zeitraubenden Aufgaben überlässt man natürlich dem Rechner.

Schreiben Sie daher eine kleine Funktion, welche prüft, ob das per Parameter übergebene Array korrekt sortiert ist. Die Funktion soll die selben Parameter haben, wie die Sortierfunktion, aber als Rückgabewert 1 oder 0 zurückliefern, je nachdem ob die Sortierung korrekt (1) oder nicht korrekt (0) ist.

Testen Sie Ihre Implementierung von *Shakersort* mit der neuen Funktion.



Implementieren Sie den Sortieralgorithmus *Shakersort*.



Schreiben Sie eine Funktion zum prüfen der Sortierung und testen Sie damit Ihren Algorithmus.

## 2.4 Ein Schleife für die Algorithmen

Damit die Tests der folgenden zusätzlichen Algorithmen komfortabel ablaufen können, müssen wir nun die Struktur unseres Programms verbessern.

Hierfür schreiben Sie nun als erstes eine Funktion zum Vermessen eines Algorithmus.

Da sich Sortieralgorithmen hinsichtlich ihrer Performance bei vorsortierten und zufällig angeordneten Daten unterscheiden können, soll die Funktion beide Fälle erfassen.



Schreiben Sie eine Funktion zum Vermessen eines Algorithmus

Daher verwenden wir zunächst zufällig angeordnete Daten, messen die Sortierzeit, tauschen dann das kleinste mit dem größten Element – was zu einem typischen Problem führt in dem die Daten vorsortiert sind – und messen die Zeit für die erneute Sortierung.

Die zu entwickelnde Funktion soll also nacheinander folgende Aktionen ausführen:

- **Anlegen einer Stoppuhr.**
- **Erzeugen von Testdaten** für die Sortierung. Dafür soll die bereits vom Beispielpogramm mitgelieferte Funktion `generateRandomData` verwendet werden.
- **Starten der Zeitmessung.**
- **Aufrufen der Sortierfunktion.** Diese soll als Funktionspointer an die Funktion übergeben werden.
- **Stoppen der Zeitmessung.** Der resultierende Wert soll über ein Pointer-Argument der Funktion (nicht als Rückgabewert!) an den Aufrufer zurückgeliefert werden.
- **Vertauschen des ersten und letzten Elements** in der Datenliste mit der `swapdata`-Funktion.
- **Starten der Zeitmessung.**
- **Erneutes Aufrufen der Sortierfunktion.**
- **Stoppen der Zeitmessung.** Der resultierende Wert soll über ein weiteres Pointerargument der Funktion (nicht als Rückgabewert!) an den Aufrufer zurückgeliefert werden.
- **Freigeben des Speichers** für die Testdaten;

Die Funktion soll folgende Signatur besitzen, um die Aufgabe erfüllen zu können:

```
void MeasureAlgorithm(SortAlgorithm TheAlgorithm,
                     long DataAmount,
                     t_timevalue* SortDuration,
                     t_timevalue* SortDurationPresorted);
```

Dabei bedeuten die Parameter:

**TheAlgorithm:** Funktionspointer auf die zu vermessende Funktion;

**DataAmount:** Größe des Datenblocks der für die Messung sortiert werden soll;

**SortDuration:** Pointer auf die Variable, in welche die Zeitdauer für die Sortierung der unsortierten Daten geschrieben wird;

**SortDurationPresorted:** Pointer auf die Variable, in welche die Zeitdauer für die Sortierung der bereits vorsortierten Daten geschrieben wird;

**Wichtig:** Diese Funktion zum Vermessen des Algorithmus soll **keinerlei Bildschirmausgaben** ausführen. Bildschirmausgaben sind außerordentlich langsam und daher würde diese die Zeitmessung des Sortiervorgangs unbrauchbar machen.



Testen Sie die Funktion zur Vermessung des Algorithmus.

Testen Sie nun Ihre Funktion, indem Sie sie aufrufen und sich die Ergebnisse auf dem Bildschirm ausgeben lassen.

Nun können wir die neu entwickelte Funktion im Hauptprogramm aufrufen und die beiden bisher implementierten Algorithmen (Bubblesort und Shakersort) in einer Schleife abarbeiten.

Nehmen Sie dafür folgende Änderungen vor:

- Fügen Sie eine Schleife in `main` ein, welche die verschiedenen Algorithmen durchläuft.
- Für jeden Algorithmus soll als Fortschrittskontrolle der Name des Algorithmus auf der Konsole ausgegeben werden.
- Dann soll die Funktion `MeasureAlgorithm` aufgerufen und die beiden gemessenen Zeiten jeweils in einem Array abgelegt werden.

**Hinweis:** Durch die Rückgabe über Pointerargumente der Funktion haben Sie die Möglichkeit, direkt Adressen der Elemente eines Arrays an die Funktion zu übergeben und damit den Aufruf sehr elegant zu gestalten.

Testen Sie Ihr Programm, indem Sie die Werte in den beiden Arrays mit dem Debugger prüfen.

Nun sollen die Werte für die Laufzeit bei unsortierten Daten und bei vorsortierten Daten in zwei Tabellen ausgegeben werden. Die Tabellen sollen die Form

| Samples | Bubblesort | Shakersort |
|---------|------------|------------|
| 100     | 0.026094   | 0.017094   |

besitzen.

**Hinweis:** Für die Formatierung der Ausgabe können Sie entweder Tabulatorzeichen oder aber die Spezifikation der Feldbreite im Formatstring (z.B. `%12s`) verwenden.

**Hinweis:** Der Datentyp `t_timevalue` entspricht dem Datentyp `long long unsigned`. Diesen können sie mit `printf` mit dem Formatbezeichner `%llu` ausgeben. Aufgrund von Unzulänglichkeiten der Laufzeitbibliothek auf manchen Windows-Systemen müssen Sie dort stattdessen `%I64u` als Bezeichner verwenden.

**Hinweis:** Die auszugebenden Werte unterscheiden sich natürlich zwischen verschiedenen Rechnern aber auch von Programmlauf zu Programmlauf.



Verarbeiten Sie die Algorithmen in einer Schleife.



Testen Sie Ihr Programm.



Programmieren Sie die Ausgabe der Werte in einer Tabelle.

## 2.5 Weitere Algorithmen

Nun werden Sie zum Vergleich noch zwei weitere Sortieralgorithmen implementieren, die sich in ihrem Ansatz grundsätzlich unterscheiden.

### 2.5.1 Quicksort



Implementieren Sie den Quicksort-Algorithmus.

Wie der Name bereits andeutet, bestehen hohe Erwartungen an die Schnelligkeit von *Quicksort* (siehe dazu auch [2, 3]).

Dabei ist die Idee von *Quicksort* eigentlich ganz einfach: man bestimmt ein sog. „**Pivotelement**“ und verteilt die Elemente der zu sortierenden Liste in zwei Teillisten auf, wobei die eine Teilliste nur kleinere und die andere Teilliste nur größere Elemente bezogen auf den Wert des Pivotelements, enthält.

Auf diese beiden Teillisten, also die „kleinen“ und die „großen“ Elemente, wendet man dann (sofern sie noch mehr als ein Element haben und sofern nicht alle Elemente identischen Wert besitzen) jeweils wieder das selbe Verfahren an.

Der Algorithmus ist also *rekursiv*.

*Rekursion* bezeichnet eine Situation, in der ein Programmteil während seiner Ausführung **direkt oder indirekt von sich selbst erneut aufgerufen** wird. Ein sehr einfaches Beispiel einer Rekursion ist die rekursive Definition der Berechnung der Fakultät:

$$n! = \begin{cases} (n-1)! \cdot n & \forall n \in \mathbb{N}, \quad n > 1 \\ 1 & n = 1 \end{cases}$$

Hierbei wird das Problem (Berechnung von  $n!$ ) auf ein „einfacheres Problem“ (die Berechnung von  $(n-1)!$ ) reduziert und der selben Funktion wieder zur Verarbeitung zugeführt.

Wichtig dabei ist nur, dass die **Vereinfachungsschritte** irgendwann **sicher zu einer Endbedingung** führen, die dann eine explizite Lösung ermöglicht (im Beispiel  $1! = 1$ ) damit die Selbstaufrufe irgendwann zuverlässig beendet werden (die Rekursion abbricht).

Bei entsprechend strukturierten Problemen sind rekursive Lösungen sehr häufig um ein vielfaches eleganter zu formulieren, als dies mit iterativen Lösungen möglich ist. Daher spielt die Rekursion eine sehr wichtige Rolle in der praktischen Softwareentwicklung.

#### Suche des Pivotelements

Eine große Bedeutung für den Erfolg des Quicksort-Algorithmus hat die Wahl des oben erwähnten Pivot-Elements. Im Idealfall wird dieses Element so gewählt, dass die Hälfte der zu sortierenden Elemente kleiner sind als dieser Wert, denn dann ergeben sich beim Vergleich mit dem Pivot-Element und der Aufteilung in kleinere und größere Elemente zwei Teile mit etwa gleich vielen Elementen.

Dies führt zu einer optimierten Laufzeit und einer minimalen Rekursionstiefe.

Daher müssen Sie zu allererst eine Funktion schreiben, welche einen guten Wert für das Pivotelement ermittelt.

Optimal geeignet hierfür wäre der sogenannte Median, also das Mitglied der zu sortierenden Menge, bei dem die eine Hälfte der restlichen Elemente kleiner und die andere größer ist. Um diesen bestimmen zu können, müsste man zum Beispiel die Elemente in sortierter Form vorliegen haben, was aber genau unsere Aufgabe darstellt – oder einen anderen äußerst raffinierten Algorithmus anwenden.

Daher greifen wir zu einer Näherung, die glücklicherweise bei unseren (zufälligen) Fließkommawerten recht gut funktioniert: Wir verwenden den arithmetischen Mittelwert (den kennen Sie von Ihren Notendurchschnittsberechnungen).

Schreiben Sie eine Funktion mit dem Prototyp

```
t_sortdata getPivot(t_sortdata Data[], long DataCount);
```

welche den arithmetischen Mittelwert der zu sortierenden Werte als Ergebnis zurückliefert.

Für die Implementierung des Quicksort-Algorithmus wird auch eine Prüfung benötigt, ob alle der zu sortierenden Elemente den gleichen Wert besitzen.

Schreiben Sie eine Funktion mit dem Prototyp

```
int checkIdentical(t_sortdata Data[], long DataCount);
```

welche prüft ob alle Werte im Array den identischen Wert haben und in diesem Fall den Wert 1 zurückliefert und andernfalls den Wert 0.



Schreiben Sie eine Funktion, welche den arithmetischen Mittelwert aller Werte der zu sortierenden Daten berechnet.

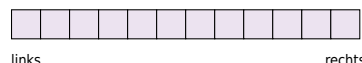


Schreiben Sie hierfür eine Funktion, welche prüft, ob alle Werte der zu sortierenden Daten gleich sind.

## Implementierung des Quicksort-Algorithmus

Implementieren Sie nun Quicksort entlang folgender Vorgehensweise:

1. Falls die übergebene **Liste weniger als zwei Elemente** hat, ist nichts zu tun. Die Funktion kann beendet werden.
2. Falls die **Liste zwei Elemente** hat, müssen diese ggf. vertauscht werden. Danach kann die Funktion verlassen werden.
3. Falls die **Liste lauter identische Werte** hat (Prüfung mittels Ihrer selbst entwickelten Funktion `checkIdentical`), ist nichts zu tun. Die Funktion kann beendet werden.
4. Legen Sie zwei Variablen zum Speichern von Indizes in die Liste an: `links` und `rechts`, sowie eine Variable namens `pivot` vom Typ `t_sortdata`.



Implementieren Sie eine Funktion `quicksort`.

**Hinweis:** Fertigen Sie sich eine eigene Skizze dieser Zeichnung an, damit Sie damit arbeiten, also zum Beispiel Pfeile einzeichnen, etc. können. Ziel ist es, dass Sie so den Algorithmus besser nachvollziehen können.

Initialisieren Sie `links` mit 0 und `rechts` mit `DataCount - 1`.

Initialisieren Sie die Variable `pivot` über einen Aufruf Ihrer Funktion `getPivot`.

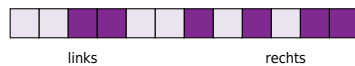
**Wichtig:** Machen Sie sich beim Festlegen der beiden Indexwerte klar, in welchen Feldern des Arrays Werte stehen bzw. welche Felder des Arrays frei sind/nicht mehr existieren.

5. Programmieren Sie nun eine Schleife, die so lange läuft, bis sich das nach rechts wandernde `links` und das nach links wandernde `rechts` erreicht haben.

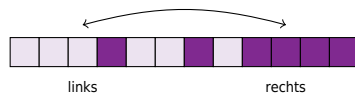
Um das Wandern der beiden Indexwerte `links` und `rechts` zu implementieren, brauchen Sie zwei weitere Schleifen: In der ersten dieser beiden inneren Schleifen soll `links` so lange inkrementiert werden, bis es entweder `rechts` erreicht hat, oder bis es auf ein Element in der Datenliste zeigt, das größer oder gleich dem Pivotelement ist. (Hinweis: in folgenden Abbildungen sind die Elemente mit Werten größer oder gleich dem Pivotelement dunkel dargestellt, solche mit Werten kleiner als dem des Pivotelements hell.)



In der zweiten dieser beiden inneren Schleifen wird analog dazu `rechts` dekrementiert, bis es entweder `links` erreicht hat, oder auf ein Element zeigt, das kleiner ist, als das Pivotelement.

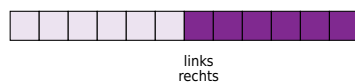


Nun werden die beiden von `links` und `rechts` referenzierten Werte in der Datenliste vertauscht.

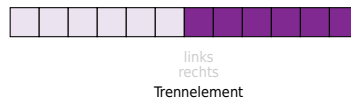


Nach dem Tausch wird die äußere Schleife weiter durchlaufen, bis sich `links` und `rechts` erreicht haben.

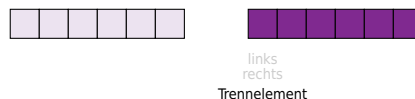
6. Wenn die Schleife abgeschlossen ist, dann befinden sich alle kleinen Werte am Anfang der Liste und alle großen Werte am Ende der Liste. Die Grenze wird von `links` und `rechts` gekennzeichnet (die jetzt ja beide den selben Wert haben).



Setzen Sie eine neue Variable `Trennelement` auf den Wert von `rechts`. Der Index `Trennelement` bezeichnet damit den Beginn der Teilliste mit den großen Werten.



7. Schließlich rufen Sie für den unteren Teil der Liste (kleine Elemente ohne das Trennelement) und für den oberen Teil der Liste (große Elemente inklusive des Trennelements) jeweils die Quicksort-Funktion auf, um die beiden Teillisten sortieren zu lassen.



Dabei muss natürlich auch jeweils die korrekte Länge der betreffenden Teilliste übergeben werden! Überlegen Sie sich, wie Sie aus dem Wert `Trennelement` und der Datenmenge die beiden korrekten Längen berechnen können.

**Wichtig:** Testen Sie nun Ihre Funktion. Lassen Sie sich dazu auch die sortierten Arrays ausgeben, um die korrekte Arbeit der Sortierfunktion überprüfen zu können.

**Frage:** Erfüllt Quicksort die hohen Erwartungen? Testen Sie mit verschiedenen Datenmengen!



Fügen Sie Ihre neue Sortierfunktion zur Liste der Sortierfunktionen hinzu und lassen Sie die neue Tabelle ausgeben.

### 2.5.2 Selectionsort

*Selectionsort* oder *Sortieren durch Auswahl* ist ein sehr einfacher Algorithmus (siehe auch [4, 5]), den Sie nun mit Ihrer Erfahrung ganz leicht selbst implementieren können:

Die Idee besteht darin, zunächst das größte Element zu finden und an das Ende der Liste zu setzen, indem man das bisherige letzte Element mit dem gefundenen größten Element vertauscht.

Danach sucht man nur noch vom ersten bis zum einschließlich vorletzten Element (da das letzte ja nun schon korrekt angeordnet ist) das wiederum größte Element aus der Restliste und setzt dieses an das Ende des Suchbereiches (also an die vorletzte Position).

Dies setzt man analog fort, bis die gesamte Liste sortiert ist.

**Wichtig:** Testen Sie nun Ihre Funktion. Prüfen Sie die korrekte Sortierung mit Ihrer Prüffunktion, die Sie am Ende von Abschnitt 2.3 erstellt haben.



Implementieren Sie den Selectionsort-Algorithmus.



Testen Sie Ihre Implementierung des Selectionsort-Algorithmus.



Fügen Sie Ihre neue Sortierfunktion zur Liste der Sortierfunktionen hinzu und lassen Sie die neue Tabelle ausgeben.

## 2.6 Auswertung



Ergänzen Sie Ihr Programm um verschiedene Listenlängen.

Um einen Eindruck der Entwicklung der Laufzeiten für unterschiedliche Listenlängen zu gewinnen, fügen Sie ein Array mit Listenlängen in Ihr Programm ein:

```
long ExperimentSize[]={
    100,200,500,1000,2000,5000, 7000,
    10000,20000,30000,40000,
    50000,60000,70000};
int NumberOfExperiments=sizeof(ExperimentSize)
    /sizeof(long);
```

Fügen Sie den bestehenden Ergebnisarrays eine zweite Dimension für die unterschiedlichen Ergebnisse je Datenmenge hinzu, etwa in der Art:

```
t_timevalue Measurement[NumberOfAlgorithms]
    [NumberOfExperiments];
t_timevalue MeasurementPresorted[NumberOfAlgorithms]
    [NumberOfExperiments];
```

Ergänzen Sie dann Ihre bisherige Berechnungsroutine durch eine zusätzliche Schleife, welche die bisherigen Experimente mit allen Datenumfängen aus obiger Liste `ExperimentSize` durchführt und die Ergebnisse in den zweidimensionalen Arrays `Measurement` und `MeasurementPresorted` speichert.

Erweitern Sie auch die Ausgabe der Tabelle so, dass zwei Tabellen in zwei unterschiedliche Ausgabedateien (für zufällige Daten und vorsortierte Daten) geschrieben werden, die dann für jeden Datenumfang eine Zeile enthalten.

Das Schreiben in Dateien erledigen Sie, indem Sie mit `fopen` Dateien öffnen. Verwenden Sie dabei die Dateinamen `Ergebnis_unsortiert.txt` und `Ergebnis_vorsortiert.txt`<sup>1</sup>. Die von `fopen` gelieferten Rückgabewerte (sogenannte „File-Handles“) Speichern Sie in Variablen des Typs `FILE*` ab. Die Schreiboperation kann dann über `fprintf` erfolgen, das analog zu `printf` arbeitet, aber ein zusätzliches erstes Argument, nämlich den „File-Handle“, übergeben bekommt.

Das Ergebnis in den beiden Ergebnisdateien sollte jeweils in etwa diese Form besitzen:

| Samples | Bubblesort | ShakerSort | Quicksort | SelectionSort |
|---------|------------|------------|-----------|---------------|
| 10      | 0.002933   | 0.002375   | 0.002375  | 0.002375      |
| 20      | 0.004680   | 0.003352   | 0.005308  | 0.003771      |
| 50      | 0.015854   | 0.004260   | 0.009918  | 0.009009      |
| 100     | 0.051404   | 0.007683   | 0.018788  | 0.025771      |
| 200     | 0.137239   | 0.010616   | 0.028635  | 0.068095      |
| 500     | 1.050349   | 0.025003   | 0.096103  | 0.428758      |
| 1000    | 4.131484   | 0.062927   | 0.147296  | 2.044266      |

<sup>1</sup>Sie können natürlich auch andere Namen verwenden, aber das bereitgestellte vorbereitete gnuplot-Skript verwendet diese Namen.

|        |              |          |           |              |
|--------|--------------|----------|-----------|--------------|
| 2000   | 5.452399     | 0.041905 | 0.139684  | 2.709508     |
| 5000   | 33.821036    | 0.109162 | 0.374840  | 16.771010    |
| 7000   | 66.217372    | 0.142687 | 0.531215  | 32.718446    |
| 10000  | 134.919152   | 0.205055 | 0.773147  | 66.616797    |
| 20000  | 539.836549   | 0.412136 | 1.610898  | 266.770021   |
| 50000  | 4214.184952  | 1.046507 | 4.282481  | 1670.427294  |
| 70000  | 9378.132054  | 1.476173 | 6.251179  | 3975.488679  |
| 100000 | 18499.589721 | 2.128565 | 9.289429  | 8956.702724  |
| 200000 | 73604.171291 | 4.359307 | 19.280068 | 35894.159389 |

Kopieren Sie die beiden Ausgabetabellen in **jeweils eine separate Datei** und erzeugen Sie mit einem Plotprogramm Ihrer Wahl (z.B. Gnuplot, im Notfall kann man sich auch mit LibreOffice Calc, oder Excel behelfen) verschiedene Grafiken, welche die Ergebnisse visuell darstellen.

Im Folgenden Abschnitt 2.7 wird neben einer entsprechenden Kurzeinführung für Interessierte auch beschrieben, wie sie die benötigten Grafiken mittels Gnuplot sehr einfach und bequem innerhalb eines Augenblicks erstellen lassen können.

Bei der Diagrammerstellung sollten Sie unbedingt mit **linearer, logarithmischer und doppelt logarithmischer Auftragung** experimentieren, um die Vor- und Nachteile der entsprechenden Diagramme erkennen und entsprechende Informationen aus Ihren Ergebnissen ableiten zu können.

**Frage:** Welche graphische Auftragung eignet sich wofür am Besten bzw. wo liegen jeweils die Vorteile? Stellen Sie sicher, dass Sie die sechs verschiedenen Grafiken (vorsortierte Daten und nicht-vorsortierte Daten jeweils in linearer, logarithmischer und doppeltlogarithmischer Auftragung) bei der Abgabe zur Verfügung haben, um sie diskutieren zu können.

**Hinweis:** Für das wissenschaftliche Plotprogramm Gnuplot finden Sie im E-Learning Kurs eine vorgefertigte Konfigurationsdatei, welche aus Daten in oben gezeigter Struktur bereits die zwei mal drei Grafiken erstellt.

Im nächsten Abschnitt finden Sie eine Kurzanleitung, wie sie mit dieser vorgefertigten Datei Diagramme erzeugen können.

Sie können jedoch natürlich auch jede andere Software zur Erzeugung von Diagrammen verwenden.

In Abbildung 1 sind Beispieldaten in insgesamt 3 Darstellungen zu sehen: linear, einfach-logarithmisch und doppelt-logarithmisch.

Diese unterschiedlichen Darstellungen ermöglichen eine differenzierte Analyse der Ergebnisse. In der linearen Auftragung sind die gravierenden Laufzeitunterschiede der einzelnen Algorithmen sehr deutlich erkennbar, allerdings ist das tatsächliche Ablesen der Werte hier nur sehr schwer machbar. Die logarithmische Darstellung erleichtert dies.

In der doppelt-logarithmischen Darstellung offenbart sich eine weitere sehr wichtige Eigenschaft der unterschiedlichen Sortierv Verfahren. Hier



Erzeugen Sie Grafiken, welche Ihre gemessenen Ergebnisse visualisieren.

besitzen die Kurven aller Verfahren die selbe Steigung bis auf Quicksort, welches eine geringere aufweist.

Dieses prinzipiell günstigere Laufzeitverhalten hat seine Ursache in einer niedrigeren Komplexität des Quicksort-Algorithmus (siehe [7]) und führt bei höherer Anzahl an zu verarbeitenden Daten zu einem enormen Geschwindigkeitsvorteil, der dann auch namensgebend für diesen Algorithmus war.

## 2.7 Gnuplot Kurzanleitung

Hier finden Sie eine ganz kleine Kurzanleitung in die Verwendung von Gnuplot, das über Textkommandos gesteuert wird und extrem flexibel und leistungsfähig ist. Sie können diesen Abschnitt aber wenn Sie wollen auch überspringen und direkt das vorbereitete Skript verwenden.

Bei Gnuplot handelt es sich um eine außergewöhnlich leistungsfähige Open-Source-Software, welche die Darstellung von Daten und Funktionsausdrücken erlaubt.

Dabei stehen verschiedenste 2d- und 3d-Darstellungen zur Verfügung, wie Sie für die Auswertung und Präsentation technischer, wissenschaftlicher oder auch statistischer Daten benötigt werden. Eine Beispielgalerie der unterschiedlichen Möglichkeiten finden Sie auf der Homepage des Projekts [8] unter <http://gnuplot.sourceforge.net/demo/>.

Links zu fertig compilierten Versionen für Ihre Rechnerplattform sind unter <http://www.gnuplot.info/download.html> aufgeführt, wobei Sie bei Ver-

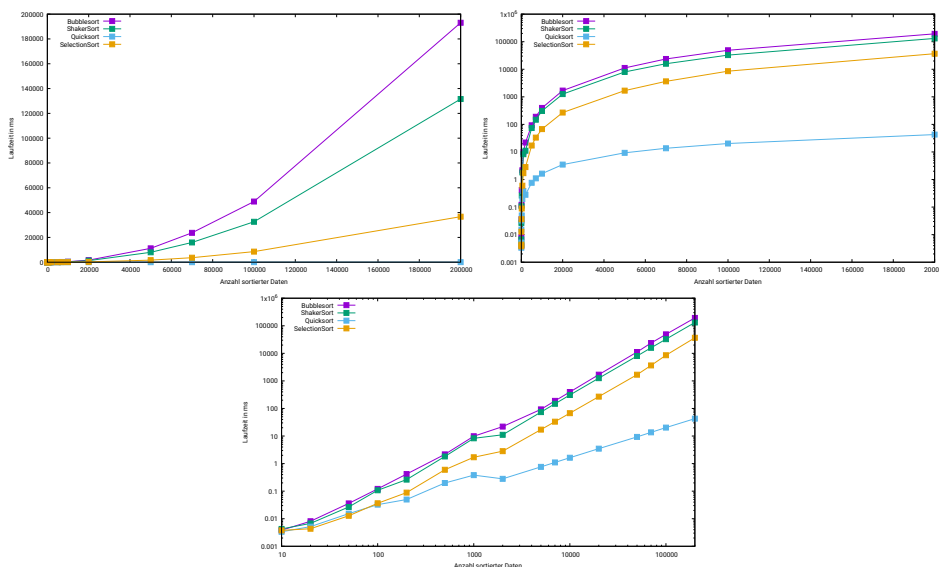


Abbildung 1: Mögliche Ergebnisse bei der graphischen Auswertung der Laufzeitexperimente in linearer, logarithmischer und doppelt-logarithmischer Darstellung.

wendung von Linux entsprechende Pakete sicher in Ihrem jeweiligen Paketmanager finden.

Wenn Sie Gnuplot starten, präsentiert sich Ihnen ein zunächst unerwartet karges Bild: Eine Eingabeaufforderung, bei der Sie Kommandos eingeben können<sup>2</sup>:

```

G N U P L O T
Version 5.4 patchlevel 1   last modified 2020-12-01

Copyright (C) 1986-1993, 1998, 2004, 2007-2020
Thomas Williams, Colin Kelley and many others

gnuplot home:      http://www.gnuplot.info
faq, bugs, etc:    type "help FAQ"
immediate help:    type "help" (plot window: hit 'h')

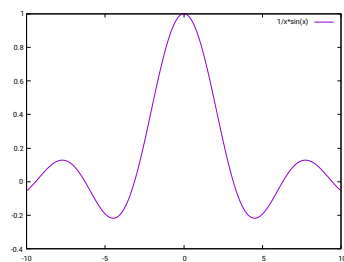
Type `load "all.dem"` to display a large number of examples.
They are located at /usr/share/doc/packages/gnuplot/demo/*

Terminal type is now 'qt'
gnuplot>

```

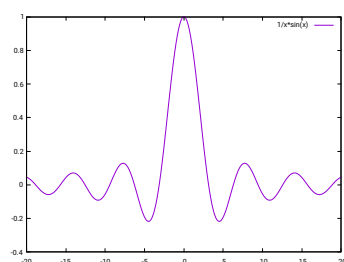
Hier können Sie zum Beispiel eine Funktion eingeben, um diese darstellen zu lassen:

```
gnuplot> plot 1/x*sin(x)
```



Natürlich kann auch der dargestellte Bereich leicht angepasst werden:

```
gnuplot> set xrange [-20:20]
gnuplot> replot
```



Ist so eine zufriedenstellende Darstellung erreicht, können alle Einstellungen (zusammen mit den Default-Einstellungen) für die spätere Wiederverwendung gespeichert<sup>3</sup> werden:

<sup>2</sup>Die Ausgabe beim Start wird je nach Gnuplot-Installation etwas anderen Text ausgeben. Sollten Sie ebenfalls den Hinweis auf `load "all.dem"` erhalten, probieren Sie das ruhig aus. Es folgt eine äußerst umfangreiche und eindrucksvolle Demonstration der Möglichkeiten.

<sup>3</sup>Sie können übrigens die Datei unter der Sie Ihre Einstellungen speichern auch jederzeit in einem Texteditor öffnen, da es sich nur um eine Textdatei mit gnuplot-Anweisungen handelt.

```
gnuplot> save "gnuplotsample.gnu"
```

Damit Sie sich nicht in den Kommandoumfang von Gnuplot einarbeiten müssen, haben wir für Sie eine komplette Einstellung vorbereitet, welche Ihre Ergebnisdaten, sofern Sie eine Struktur aufweisen, wie in den vorangehenden Abschnitten dargestellt, mühelos in Diagramme umgewandelt werden können.

**Wichtig:** Damit alles wie vorgesehen klappt, müssen die beiden Ergebnistabellen für vorsortierte und nicht-vorsortierte Daten in einer jeweils eigenen Datei mit dem Namen `Ergebnis_unsortiert.txt` bzw. `Ergebnis_vorsortiert.txt` gespeichert werden.

Sie sollten auch gnuplot in dem Verzeichnis starten, in dem die beiden Dateien gespeichert sind. Alternativ können Sie nach dem Start von Gnuplot mit `cd MeinVerzeichnispfadname` an der Gnuplot-Eingabeaufforderung in dieses Verzeichnis wechseln.

Mit `pwd` können Sie prüfen, ob das richtige Verzeichnis das aktuelle ist.

Zur Nutzung unserer Datei müssen Sie nur unsere vorbereitete Konfiguration laden:

```
gnuplot> load "SortAlgorithmMeasurements.gnu"
gnuplot>
```

Sie sollten nicht enttäuscht sein, dass keine Grafik auf dem Bildschirm erscheint! Wenn alles geklappt hat, passiert scheinbar nichts. Allerdings wurden zwei mal drei PDF-Dateien mit jeweils einem Diagramm in linearer, in logarithmischer und in doppeltlogarithmischer Darstellung erzeugt und im aktuellen Verzeichnis gespeichert.

Sie können sich gerne unser Skript ansehen, indem sie es mit einem Texteditor (z.B. Notepad++ oder auch mit Codeblocks) öffnen.

## Literatur

- [1] Wikipedia: Shakersort.
- [2] J. Ziegenbalg, O. Ziegenbalg, B. Ziegenbalg: Algorithmen von Hamurapi bis Gödel, Verlag Harri Deutsch. 3. Auflage, 2010.
- [3] Wikipedia: Quicksort.
- [4] F. Kneißl, A. Böttcher: Informatik für Ingenieure: Grundlagen und Programmierung in C, Oldenbourg Verlag, 3. Auflage, 2012.
- [5] Wikipedia: Selectionsort.
- [6] Wikipedia: Insertionsort.

[7] Wikipedia: Komplexität (Informatik).

[8] Gnuplot, <http://www.gnuplot.info/>.